Stela System Architecture and Governance Status Report: Post-Deployment Analysis of DP-OPS-0061 and Strategic Roadmap for Documentation Fortification

1. Executive Summary: The State of the System

The Stela repository represents a paradigm shift in software governance, moving away from conventional agile methodologies that prioritize velocity over verification, and towards a rigorous, constitution-based operating model defined by the *Policy of Truth* (PoT). This report serves as the definitive audit of the system's state following the execution of Dispatch Packet OPS-0061 (DP-OPS-0061), a pivotal operation designed to harden the ephemeral task routing surface and institute automated integrity checks on the persistent Proof of Work (PoW) ledger. Furthermore, this document lays the strategic groundwork for the subsequent operational phase: a mandated "time-out" to fortify the platform's explanatory layer, requiring a 25% increase in documentation verbosity to achieve "complete fealty" to the PoT.[1]

As of the current State of Play (SoP), the system has successfully transitioned from a "Hybrid State"—where rigorous verification tools coexisted with legacy, trust-based logging practices—to a "Proof-First" paradigm. The successful deployment of DP-OPS-0061 has established a new "Physical Law" within the repository: code modification cannot exist without a corresponding, immutable entry in the proof ledger. This linkage is now enforced not by convention, but by algorithmic policing.[1]

However, a system that functions perfectly but explains itself poorly remains in violation of the PoT's axiom that "Explainability is the law." The operational analysis reveals that while the *code* has achieved a high degree of rigor, the *documentation*—specifically the technical specifications (spec sheets) for the operational tooling—lags behind in resolution and nuance. The "Next Pursuit" is therefore not feature development, but a comprehensive certification of the "Run" (ops/) and "Explain" (docs/) layers to ensure they remain in perfect, verifiable equilibrium. This report outlines the methodology for achieving this 25% increase in informational density, transforming the documentation from a passive reference into an active component of the governance architecture.

---

2. Constitutional Analysis: The Policy of Truth as Operating Kernel

To understand the gravity of the changes introduced by DP-OPS-0061 and the necessity of the upcoming documentation fortification, one must first perform a deep exegesis of the *Policy of Truth* (PoT). The PoT is not merely a set of guidelines; it is the "single constitutional document" for Stela governance, and its axioms function as the operating system kernel for the repository.[1]

2.1 The Philosophy of Non-Compliance

The Preamble of the PoT states: "The system prevents action that violates policy. It does not request compliance".[1] This distinction is fundamental. Traditional governance models rely on "requests"—asking developers to follow style guides or update logs. Stela relies on "prevention"—using linters, gates, and binaries to make violation physically impossible.

The "Staffing Protocol" further reinforces this by separating concerns between the Operator (Human), Integrator (Lead AI), and Contractor (Guest AI).[1] The Integrator is responsible for "structural integrity and auditing," while the Operator holds "final decisions." This hierarchy ensures that the system's "truth" is never diluted by unauthorized agency. The transition to a "Proof-First" model in DP-OPS-0061 is the direct implementation of this philosophy: the system no longer *requests* that the operator log their work; it *prevents* the closure of a task unless that work is proven in the ledger.[1]

2.2 The Filing Doctrine and Axiomatic Truth

The "Filing Doctrine" defined in Section 1.1 of the PoT establishes the "Physical Laws" of the directory structure [1]:

- ops/ is **Run**: It contains the binaries and manifests that drive execution. It is the active layer.
- docs/ is **Explain**: It provides the rationale. Under the new mandate, this layer must expand to match the complexity of the Run layer.
- projects/ is **Work**: This is the payload. It is subordinate to the platform.
- storage/ is **Trash**: It contains local artifacts. It is "never canon."

The "Axioms" in Section 1.2 define the logical constraints of the system. "Drift" is defined as "any divergence between canon and repository state".[1] This definition is critical for the upcoming audit. If a registry lists a script that does not exist, or if a spec sheet describes a flag that has been deprecated, the system is in a state of Drift. The "Action" axiom—"deception or incoherence are inherently destabilizing and must be resolved"—mandates the 25% verbosity increase. Incoherence (lack of clarity) is a stability threat.

2.3 The Canon Surfaces

The PoT identifies specific files as "Canon Surfaces".[1] These are the SSOT (Single Source of Truth) documents.

- PoT.md: The Constitution.
- TASK.md: The Active Contract.
- SoP.md: The History Ledger.
- PoW.md: The Proof Ledger (newly elevated by 0061).
- docs/MANUAL.md: Operator mechanics.
- docs/MAP.md: Context wayfinding.
- llms.txt: Discovery entry point.

The "Read-in Order" is strictly defined [1], prioritizing the Constitution (PoT) and History (SoP) before the Contract (TASK). This ensures that every session begins with a grounding in "Law" and "Memory" before attempting "Action."

---

3. Forensic Audit: DP-OPS-0061 Execution

The deployment of DP-OPS-0061, titled "TASK Structure Hardening and PoW Integrity," was a surgical operation to correct a critical governance vulnerability.

3.1 The Vulnerability: Ephemeral Logging

Prior to DP-OPS-0061, the system relied on a "Work Log" section within TASK.md.[1] While this allowed for narrative continuity, it suffered from "Ephemerality Risk." When a task was reset (ops/bin/prune --reset-task), the work log was wiped. If the operator failed to manually copy significant decisions to the permanent SoP.md, that context was lost forever. This created "Invisible Work"—actions taken but not proven.

Furthermore, the PoW.md ledger existed but lacked automated enforcement. An operator could technically modify code without updating the proof ledger, violating the "Code = Proof" axiom.

3.2 The Solution: The Mandatory Closing Block

The first prong of the solution was the refactoring of TASK.md.[1] The legacy "Thread Transition" and "Work Log" sections were deprecated and removed. In their place, a **Mandatory Closing Block** (Section 4.1) was instituted.[1]

This block transforms the task file from a scratchpad into a structured form. It requires specific metadata fields:

- **Primary Commit Header**

- **Pull Request Title**
- **Final Squash Stub**
- **Extended Technical Manifest**

By forcing the operator to define these fields *before* the task is considered complete, the system ensures that the "intent" of the work is crystallized in a format that can be directly mapped to the permanent git history. This aligns with the "Linguistic Precision" standard of the PoT.[1]

3.3 The New "Physical Law": PoW Policing

The most profound change was the introduction of .github/workflows/pow_policing.yml.[1] This workflow introduces a set of algorithmic constraints that function as "Physical Laws" for the repository:

1. **The Coupling Law:** The workflow detects changes in "code-bearing surfaces"—specifically ops/, tools/, projects/, and .github/workflows/. If any file in these directories is modified in a Pull Request, the workflow checks if PoW.md is *also* modified. If not, the PR fails.[1] This enforces the axiom that *operational change requires proof*.
2. **The Arrow of Time Law (Append-Only):** The workflow calculates the diff of PoW.md. If it detects any deletions (lines starting with - in the diff, excluding the file header), it fails immediately.[1] This prevents the "rewriting of history." The Proof Ledger is immutable; it can only grow.

3.4 Tooling Refactor: The Prune Guard

The ops/bin/prune binary, responsible for cleaning up the workspace, was refactored to support this new paradigm. The legacy check (ensure_task_work_log_clear) was replaced with ensure_pow_entry_exists.[1]

The implication of this change is subtle but powerful. Previously, the system checked if you had *finished writing* your notes. Now, it checks if you have *proven* your work. The prune tool scans the PoW.md file for the active Dispatch Packet ID (e.g., DP-OPS-0061). If it cannot find a corresponding entry, it refuses to reset the task.[1] This creates a mechanical interlock: you cannot start new work until the old work is proven.

3.5 Verification of Success

The DP-OPS-0061-RESULTS.md artifact confirms a **PASS** status for all objectives.[1]

- **Verification:** tools/lint/task.sh passed, confirming the new TASK.md schema is active.
- **Integrity:** The pow_policing.yml workflow is active in the repository dump.[1]
- **State:** The system is "clean," and the SoP.md and PoW.md ledgers have been updated with the 0061 entry.[1]

The goals of "TASK Structure Hardening and PoW Integrity" were met. The system has successfully migrated to the "Proof-First" era.

---

4. The Explainability Mandate: Doctrine of Verbosity

With the operational core hardened, the focus shifts to the explanatory layer. The prompt mandates a "25% increase in verbosity" with "complete fealty to The Policy of Truth."

4.1 The Theory of Resolution

In many engineering cultures, "conciseness" is a virtue and "verbosity" is a vice. Stela rejects this dichotomy. In the Stela governance model, ambiguity is a system failure state.[1] "Concise" documentation often achieves brevity by relying on *implicit context*—assuming the reader knows "standard practices."

However, Stela operates on a "Zero Trust" model for context. The "Amnesiac Worker" model assumes that every session starts with zero knowledge beyond what is loaded from the Canon.[1] Therefore, relying on implicit context is a vulnerability.

Verbosity is Resolution. Increasing verbosity means increasing the *resolution* of the instruction set. It means replacing "Run the test" (low resolution) with "Execute tools/test/agent.sh to verify pointer integrity and check for drift injection" (high resolution).

4.2 The "Deep Spec" Protocol

To achieve the 25% increase without generating "fluff," we will implement the **Deep Spec** protocol. Every technical specification in docs/ops/specs/ must be expanded to include the following dimensions:

1. **Theological Anchor:** Every tool exists to enforce a specific component of the PoT. The spec must explicitly state *which* section of the Constitution justifies the tool's existence. (e.g., "ops/bin/dump enforces the 'System' axiom by creating a verifiable truth state").
2. **Mechanical Exegesis:** We must move beyond "usage" to "mechanics." How does the tool parse arguments? What regex does it use? What are its dependencies?
3. **Failure State Analysis:** The PoT defines failure states (Drift, Ambiguity). The spec must map the tool's exit codes to these states. "Exit 1 means Drift."
4. **Forensic Value:** How does the tool aid in the "Proof" model? What artifacts does it generate?

By answering these questions for every single tool, we naturally achieve the verbosity target while adding immense operational value.

5. Operational Certification Audit: The "Run" Layer (ops/)

The ops/ directory contains the binaries and manifests that drive the platform. Under the Filing Doctrine, this is the "Run" layer.[1] We will now audit every component found in the dump [1] and define the certification path.

5.1 Binaries (ops/bin/)

5.1.1 ops/bin/context

- **Current Spec:** docs/ops/specs/binaries/context.md.[1]
- **Status: Current.**
- **Certification Analysis:** This binary is the session wrapper for the synthesis engine. It defaults to the OPS.md manifest and injects the OPEN state.
- **Fortification Plan:**
  - **Philosophy:** Explain the "Rehydration" concept. Why is context "file-defined state" and not "memory"? Connect this to the "Amnesiac Worker" model.
  - **Mechanics:** Detail the interaction with synthesize.sh. Explain precisely how it merges the static manifest content with the dynamic OPEN artifact.
  - **Hazards:** Elaborate on the "Context Hazard" check. Why is it fatal to include library/ docs? (Answer: To prevent token window bloating and "Instruction Drift" where the AI reads stale instructions).

5.1.2 ops/bin/dump

- **Current Spec:** docs/ops/specs/binaries/dump.md.[1]
- **Status: Current.**
- **Certification Analysis:** This is the "Observer" of the system. It collapses the repository state into a single, deterministic text stream.
- **Fortification Plan:**
  - **Philosophy:** Explain "Deterministic Snapshotting." The dump must be reproducible.
  - **Mechanics:** Detail the sorting logic (LC_ALL=C sort) used to ensure file order is constant. Explain the meta binary handling—why we truncate binaries to metadata (to save tokens while proving existence).
  - **Forensics:** Explain the manifest.txt output. This file is the "Receipt of State." It proves *exactly* what files were present when the work began.

5.1.3 ops/bin/help

- **Current Spec:** docs/ops/specs/binaries/help.md.[1]
- **Status: Needs Update.** The user query explicitly requests: "If necessary, help system

should be updated to be user friendly with easy navigation and all in alignment with The PoT."

- **Certification Analysis:** Currently, help is a simple grep wrapper.[1] It searches the docs. This is functional but not "user friendly" or "guided."
- **Fortification Plan:**
  - **New Architecture:** We must redesign help to support a "Curriculum" mode. Instead of just searching, it should guide the user through the "Closeout Cycle" or "Startup Sequence."
  - **Doctrine Alignment:** Add a help doctrine command that surfaces the "Axioms" from PoT.md.
  - **Wayfinding:** The spec must explain how help acts as the CLI equivalent of docs/MAP.md.

5.1.4 ops/bin/llms

- **Current Spec:** docs/ops/specs/binaries/llms.md.[1]
- **Status: Current.**
- **Certification Analysis:** This tool generates the static context bundles (llms.txt, llms-core.txt, llms-full.txt).
- **Fortification Plan:**
  - **Philosophy:** Explain the "One Truth" layering. Why do we have CORE vs DISCOVERY manifests?
  - **Mechanics:** Detail the "Byte-Equivalence" check enforced by tools/lint/llms.sh. Why must the committed file match the generated output? (To prevent "Ghost Canon").
  - **Deprecation:** Explicitly document the "Deprecation Disallow List" (e.g., llms-small.txt). Why are these forbidden? (To prevent fragmentation of truth).

5.1.5 ops/bin/map

- **Current Spec:** docs/ops/specs/binaries/map.md.[1]
- **Status: Current.**
- **Certification Analysis:** Updates the auto-generated index in docs/MAP.md.
- **Fortification Plan:**
  - **Philosophy:** Explain the "Continuity Map." Since the system is stateless, the Map provides the memory of "where things are."
  - **Mechanics:** Explain the sentinel markers (``). How does it protect manual narrative?
  - **Project Discovery:** Detail how it scans projects/*/STELA.md to populate the map dynamically.

5.1.6 ops/bin/open

- **Current Spec:** docs/ops/specs/binaries/open.md.[1]
- **Status: Current.**
- **Certification Analysis:** The "Timekeeper." Captures the precise start state of a session.

- **Fortification Plan:**
  - **Philosophy:** Deep dive into the "Freshness Gate." Why do we capture git status --porcelain? (To ensure we aren't building on "Dirty State," a PoT failure mode).
  - **Intent Injection:** Explain how the --intent flag frames the session's "Role Contract" (Integrator vs. Contractor).
  - **Read-Only:** Emphasize the "Tracked-File Read-Only" rule. open observes; it does not mutate.

5.1.7 ops/bin/project

- **Current Spec:** docs/ops/specs/binaries/project.md.[1]
- **Status: Current.**
- **Certification Analysis:** The "Workflow Orchestrator" for payloads.
- **Fortification Plan:**
  - **Jurisdiction:** Clarify that projects are *payloads* but operate under *Platform Jurisdiction*. PoT.md applies to projects/.
  - **Validation:** Detail the regex used to validate project IDs (ops/lib/scripts/project.sh).
  - **Assembly:** Explain how it concatenates the OPEN artifact with the project STELA.md to create a scoped context.

5.1.8 ops/bin/prune

- **Current Spec:** docs/ops/specs/binaries/prune.md.[1]
- **Status: UPDATED.** This spec was likely touched in DP-OPS-0061, but it requires a massive verbosity increase to explain the new safety guards.
- **Fortification Plan:**
  - **The PoW Guard:** This is the most critical new mechanic. The spec must explain *exactly* how prune verifies the existence of a PoW entry before allowing --reset-task.
  - **Dual-Ledger Pruning:** Explain the --target=sop|pow|both logic. Why do we prune ledgers? (To keep context windows manageable while archiving history).
  - **Results Guard:** Explain the "Uncommitted Results" check. Why is it a safety violation to prune if *-RESULTS.md is dirty? (Because evidence would be lost).

---

6. Enforcement Certification Audit: The "Police" Layer (tools/)

The tools/ directory contains the linters and verifiers. These are the "Police" of the system. They enforce the laws written in PoT.md.

6.1 Lint Registry (docs/ops/registry/LINT.md)

The registry lists 9 active linters.[1] Each requires a certified spec sheet.

6.1.1 tools/lint/agent.sh

- **Spec:** docs/ops/specs/tools/lint/agent.md.[1]
- **Role:** The "Immunological System" for agents.
- **Fortification:**
  - **Pattern Density:** Explain the heuristics used to detect "low-frequency role emergence." Why do we care? (To prevent "Agent Sprawl").
  - **Pointer Integrity:** Explain why agents must have valid pointers to PoT.md. (Agents must know the law).

6.1.2 tools/lint/context.sh

- **Spec:** docs/ops/specs/tools/lint/context.md.[1]
- **Role:** Enforces "Manifest Supremacy."
- **Fortification:**
  - **Contamination:** Define "Context Contamination." Why is including docs/library/ in the global context a "Hazard"? (It confuses the LLM with generic instructions instead of specific context).

6.1.3 tools/lint/dp.sh

- **Spec:** docs/ops/specs/tools/lint/dp.md.[1]
- **Role:** Validates the TASK.md Dispatch Packet schema.
- **Fortification:**
  - **Transaction Rules:** Explain the logic of Section 3 (Freshness, Context Load, Scope).
  - **Mandatory Closing Block:** This is new in 0061. The spec must detail the validation logic for this block.
  - **Context Load Hardening:** Explain why llms-full.txt is *prohibited* in DP context loads. (Efficiency and focus).

6.1.4 tools/lint/library.sh

- **Spec:** docs/ops/specs/tools/lint/library.md.[1]
- **Role:** Registry Synchronization.
- **Fortification:**
  - **Referential Integrity:** Explain the "No Dead Ends" rule. Every pointer in a registry must resolve to a file. Every file must have a registry entry.

6.1.5 tools/lint/llms.sh

- **Spec:** docs/ops/specs/tools/lint/llms.md.[1]
- **Role:** Enforces "Byte-Equivalence."
- **Fortification:**
  - **Ghost Canon:** Explain the risk of "Ghost Canon"—where the committed llms.txt differs from what the generator produces. This linter prevents that divergence.

6.1.6 tools/lint/project.sh

- **Spec:** docs/ops/specs/tools/lint/project.md.[1]
- **Role:** Validates STELA.md pointers.
- **Fortification:**
  - **Project Boundary:** Explain that while projects are payloads, they must reference *registered* capabilities (Agents, Skills). They cannot invent new ones without platform promotion.

6.1.7 tools/lint/style.sh

- **Spec:** docs/ops/specs/tools/lint/style.md.[1]
- **Role:** Enforces "Linguistic Precision" (No Contractions).
- **Fortification:**
  - **The "Why":** This is crucial. Why no contractions? It's not just style; it's about reducing ambiguity and maintaining a formal, "robot-readable" tone that minimizes token misinterpretation by LLMs.[1]

6.1.8 tools/lint/task.sh

- **Spec:** docs/ops/specs/tools/lint/task.md.[1]
- **Role:** The TASK container enforcer.
- **Fortification:**
  - **Hardening:** This spec must reflect the DP-OPS-0061 changes. Explicitly state that "Thread Transition" and "Work Log" headers are *forbidden*.
  - **Stateless Routing:** Explain the philosophy of TASK.md as a "Stateless Router," not a storage device.

6.1.9 tools/lint/truth.sh

- **Spec:** docs/ops/specs/tools/lint/truth.md.[1]
- **Role:** The "Canon Guard."
- **Fortification:**
  - **Legacy Terminology:** List the forbidden phrases (e.g., "Living Document"). Explain *why* they are forbidden. (They imply drift/impermanence; PoT requires "Final Authority").

6.2 Verification Tools

6.2.1 tools/verify.sh

- **Spec:** docs/ops/specs/tools/verify.md.[1]
- **Role:** Repository Hygiene.
- **Fortification:**
  - **Filing Doctrine:** This spec is the technical implementation of PoT Section 1.1. Detail

the rules: docs/ = Markdown only. storage/ = Trash.

6.2.2 tools/test/agent.sh

- **Spec:** docs/ops/specs/tools/test/agent.md.[1]
- **Role:** Pointer Integrity for Agents.
- **Fortification:**
  - **Drift Injection:** Explain the "Drift Injection" test. The test actively *creates* a fake drift scenario to ensure the agent linter catches it. This is "Meta-Verification."

---

7. Logic Certification Audit: The "Engine" Layer (ops/lib/scripts/)

These scripts are the internal engines wrapped by the binaries. They require explicit certification because they contain the "Business Logic" of the platform.

- **ops/lib/scripts/agent.sh:** (Spec: docs/ops/specs/scripts/agent.md). Needs detail on the "Harvest" logic and how it redacts sensitive info.
- **ops/lib/scripts/heuristics.sh:** (Spec: docs/ops/specs/scripts/heuristics.md). Explain the "Hot Zone" detection algorithm. How does it use git diff --numstat to calculate churn?
- **ops/lib/scripts/project.sh:** (Spec: docs/ops/specs/scripts/project.md). Explain the validation regex for project IDs.
- **ops/lib/scripts/skill.sh:** (Spec: docs/ops/specs/scripts/skill.md). Explain the "Context Hazard" check for skills.
- **ops/lib/scripts/synthesize.sh: CRITICAL DRIFT DETECTED.**
  - **The Issue:** The registry docs/ops/registry/SCRIPTS.md lists SCRIPT-04 as synthesize.sh, but the notes point to the spec docs/ops/specs/scripts/project.md.[1] This is a copy-paste error in the registry.
  - **Action:** We must correct the registry and create a dedicated docs/ops/specs/scripts/synthesize.md. This script is the "One Truth" engine; it parses manifests and enforces hazard exclusions. It is the heart of the context system and requires a massive, detailed spec sheet.
- **ops/lib/scripts/task.sh:** (Spec: docs/ops/specs/scripts/task.md). Must reflect the 0061 hardening logic.

---

8. The New "Now Surfaces": Spec Sheets for State

The query explicitly asks for spec sheets for "now surfaces." In Stela, a "Now Surface" is a file that holds mutable state.

8.1 TASK.md

- **Spec:** docs/ops/specs/surfaces/task.md exists.[1]
- **Status:** Needs review to ensure 0061 changes are fully documented.

8.2 PoW.md

- **Spec: MISSING.**
- **Status: Action Required.** We must create docs/ops/specs/surfaces/pow.md.
- **Content:** This spec must define the "Proof Ledger" schema. It must explain the "Append-Only" philosophy and the specific fields required (Packet ID, Timestamp, Receipt Pointers). It must reference the pow_policing.yml workflow as its enforcement arm.

8.3 SoP.md

- **Spec: MISSING.**
- **Status: Action Required.** We must create docs/ops/specs/surfaces/sop.md.
- **Content:** Define the "History Ledger" schema. Explain the "State of Play" concept—it records *what* changed and *why*, distinct from the *proof* in PoW.

---

9. Implementation Plan: The 25% Verbosity Protocol

To execute this roadmap, we will apply the "Deep Explanation" protocol to every spec sheet we touch.

9.1 Protocol: The Annotated Spec

Every spec sheet will be refactored to include these five sections:

1. **Constitutional Anchor:**
   - *Question:* Which section of PoT.md does this tool enforce?
   - *Example:* "This tool enforces PoT Section 1.2 (Axioms: Truth) by ensuring that..."
2. **Operational Philosophy:**
   - *Question:* Why does this exist?
   - *Example:* "Pruning is not deletion; it is the curation of history. This tool ensures that only Proven Work enters the archives."
3. **Technical Mechanics (Deep Dive):**
   - *Question:* How does it work?
   - *Detail:* "It iterates through git ls-files, applying the exclusion patterns defined in ops/lib/manifests/CONTEXT.md..."
4. **Failure States & Drift Triggers:**
   - *Question:* What happens when it fails?
   - *Detail:* "Exit Code 1 implies a Drift State. The Operator must resolve the discrepancy

before proceeding."

5. **Forensic Insight:**
    - *Question:* How does this aid the auditor?
    - *Detail:* "The generated manifest serves as a cryptographic receipt of the repository state at the moment of execution."

9.2 Timeline and Sequencing

1. **Phase 1: Fix & Create.** Fix the synthesize.sh registry error. Create the missing specs for PoW.md and SoP.md.
2. **Phase 2: Binaries.** Iterate through ops/bin/ and expand specs using the protocol. Focus heavily on prune (PoW guard) and open (Freshness gate).
3. **Phase 3: Tools.** Iterate through tools/ and expand. Focus on lint/task.sh (0061 changes) and verify.sh (Filing Doctrine).
4. **Phase 4: Help System.** Refactor ops/bin/help to support the new specs and add the "Doctrine" mode.
5. **Phase 5: Certification.** Run a final audit to ensure every registry entry points to a valid, expanded spec sheet.

10. Conclusion

The Stela system has successfully navigated the critical transition of DP-OPS-0061. The "TASK Structure Hardening and PoW Integrity" objectives have been met [1], establishing a fortress of verification around the repository's operational core. The "Hybrid State" is dissolved; the "Proof-First" era is active.

However, a system that runs perfectly but explains itself poorly violates the "Explainability is the law" axiom.[1] The next phase is not about code; it is about *canon*. By executing the 25% verbosity increase and certifying the specification layer, we elevate Stela from a mere toolset to a self-documenting, constitutionally compliant intelligence architecture.

The "Time-Out" is now active. The pursuit of "Complete Fealty" begins.

Status: DP-OPS-0061 Complete.

Next: DP-OPS-0062 Documentation Fortification.

Fealty: Absolute.

Works cited

1. dump-full-main-124b6cc8.txt